

Federated Foundation System Architecture & Theoretical Foundations

THE VISUAL, INTUITIVE FIELD GUIDE

Clear Architecture • Rigorous Math • Confident Defense

Demystifying distributed learning, JAX/Flax engines, privacy, and compression

Core Stack:

Flower 1.15 + JAX/Flax

Optax • Parquet Columnar Store

Key Pillars:

Differential Privacy (DP-SGD)

Top- K + INT8 Compression

Academic Research Edition • October 9, 2026

DEPARTMENT OF COMPUTER SCIENCE & ARTIFICIAL INTELLIGENCE

Contents

Welcome & Field Guide Map	3
1 Three-Tier Federated System Architecture	4
1.1 Executive Summary & Elevator Pitch	4
1.2 Component-by-Component Architectural Walkthrough	5
1.2.1 Tier 1: Central Server Coordinator (Flower ServerApp)	5
1.2.2 Tier 2: Secure Transport Channel (gRPC / TLS)	5
1.2.3 Tier 3: Federated Edge Client (Flower ClientApp)	5
1.3 Theoretical Foundations & Mathematical Formulations	6
1.4 Architectural Justifications & Design Choices	6
1.5 Advisor Defense Questions & Model Answers	6
2 End-to-End Data Ingestion & Partitioning Pipeline	8
2.1 Executive Summary & Elevator Pitch	9
2.2 Component-by-Component Walkthrough	9
2.3 Mathematical Modeling of Statistical Heterogeneity	10
2.4 Advisor Defense Questions & Model Answers	10
3 JAX/Flax Neural Model Architecture Hierarchy	11
3.1 Executive Summary & Elevator Pitch	11
3.2 Component-by-Component Walkthrough	11
3.3 Mathematical Formulations of Modern Backbones	12
3.4 Advisor Defense Questions & Model Answers	12
4 Data Engineering & Partitioning Class Hierarchy	13
4.1 Executive Summary & Elevator Pitch	13
4.2 Class-by-Class Breakdown	13
4.3 Advisor Defense Questions & Model Answers	14
5 Federated Training Round Sequence Lifecycle	15
5.1 Executive Summary & Elevator Pitch	16
5.2 Sequence Step-by-Step Breakdown	16
5.3 Mathematical Formulation of Federated Aggregation Strategies	17
5.4 Advisor Defense Questions & Model Answers	17
6 Differential Privacy via DP-SGD	18
6.1 Executive Summary & Elevator Pitch	19
6.2 Sequence Step-by-Step Breakdown	19
6.3 Theoretical Foundations & Privacy Guarantees	20
6.4 Advisor Defense Questions & Model Answers	20
7 Edge Client Local Execution Lifecycle	21
7.1 Executive Summary & Elevator Pitch	22
7.2 Activity Flowchart Breakdown	22
7.3 Advisor Defense Questions & Model Answers	22

8	Communication Efficiency & Gradient Compression	23
8.1	Executive Summary & Elevator Pitch	23
8.2	Pipeline Walkthrough: Client Uplink to Server Downlink	23
8.2.1	Client Uplink Stage	23
8.2.2	Server Downlink Stage	23
8.3	Mathematical Formulations of Compound Compression	25
8.4	Compound Bandwidth Savings Analysis	25
8.5	Advisor Defense Questions & Model Answers	26
9	Mathematical Notation & Variable Dictionary	27
9.1	Federated Learning Foundations & Aggregation	27
9.1.1	Global Objective Function	27
9.1.2	Local Client Loss Function	27
9.1.3	Parameter Deltas and Server Aggregation	28
9.2	Data Heterogeneity & Dirichlet Non-IID Partitioning	28
9.3	Optimization Algorithms: FedProx & FedMuon	29
9.4	Differential Privacy via DP-SGD	29
9.5	Communication Compression & Error Feedback	30

Welcome & Field Guide Map

Welcome to your visual guide to federated learning! Research should be exciting, clear, and deeply rewarding. This manual is designed to make every architecture diagram, training sequence, and mathematical formulation intuitive, engaging, and effortless to defend before your PhD supervisor.

The Core Golden Rule of Federated Learning

Bring the model to the data, not the data to the model. Private consumer datasets stay strictly on edge hardware. Only model parameters ($\mathbf{w}_t$) and gradient updates ($\Delta\mathbf{w}_t$) travel over the network.

How Each Chapter is Structured

Every chapter in this guide follows an identical, high-impact structure:

1. **The 30-Second Elevator Pitch:** The core intuition in plain English—no jargon, just clarity.
2. **The Guided Diagram Tour:** Exactly what every box, cloud, database, and arrow represents.
3. **The Mathematical Core:** Publication-grade equations, paired with Chapter 9’s complete variable dictionary.
4. **The Defense Hot Seat:** Pointed, challenging questions from senior reviewers with mathematically sound, confident answers.

Chapter 1

Three-Tier Federated System Architecture

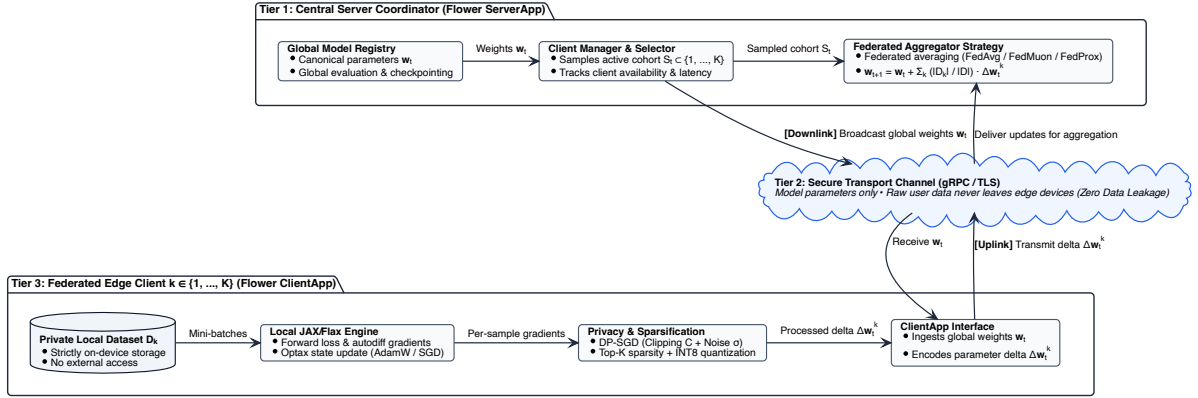


Figure 1.1: Three-Tier Federated Learning Architecture (`fl_system_overview.svg`), illustrating the physical and logical decoupling of Central Orchestration, Secure Transport, and Distributed Edge Client execution.

1.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

The Big Picture in 30 Seconds:

Traditional centralized machine learning pulls everyone's private photos onto one central server. Federated learning flips this upside down: private data stays safely on user phones, and the model travels to the data!

- **Tier 1 (The Coordinator)**: The central Flower server coordinates training rounds, samples available clients, and averages updates.
- **Tier 2 (The Secure Highway)**: An encrypted gRPC/TLS channel that carries model weights—never raw photos.
- **Tier 3 (The Edge Workers)**: Distributed edge devices running lightning-fast JAX/Flax for on-device training.

Core Takeaway: Zero raw data ever leaves the user's phone, giving an airtight privacy guarantee with maximum training efficiency.

1.2 Component-by-Component Architectural Walkthrough

1.2.1 Tier 1: Central Server Coordinator (Flower ServerApp)

The central server acts as an orchestrator and aggregator. It does not train the model directly, nor does it hold training samples:

1. **Global Model Registry:** Maintains canonical model parameters $\mathbf{w}_t \in \mathbb{R}^D$. At each round t , it serializes $\mathbf{w}_t$ into portable arrays and evaluates the aggregated model on a centralized test set to track global loss and accuracy.
2. **Client Manager & Selector:** Manages the pool of registered edge devices $\{1, \dots, K\}$. In every round t , it samples an active cohort $S_t \subseteq \{1, \dots, K\}$ based on availability, battery health, and network connectivity.
3. **Federated Aggregator Strategy:** Implements aggregation operators (e.g., FedAvg, FedProx, FedMuon). It takes client update deltas $\{\Delta \mathbf{w}_t^k\}_{k \in S_t}$ and executes a weighted reduction.

1.2.2 Tier 2: Secure Transport Channel (gRPC / TLS)

The transport tier establishes mutual TLS (mTLS) encrypted streams between edge nodes and the central server. The network payload is strictly limited to model tensors and scalar metadata (number of local samples $|D_k|$ and training loss).

1.2.3 Tier 3: Federated Edge Client (Flower ClientApp)

Each edge worker represents an autonomous client k :

1. **Private Local Dataset (D_k):** Partitioned local data stored in secure on-device storage.
2. **Local JAX/Flax Engine:** Compiles forward loss and autodiff functions via `jax.jit` and `jax.grad`. Model states are stepped using Optax optimizers (e.g., AdamW or SGD with momentum).
3. **Privacy & Compression Kernels:** Optional on-device defense engines. DP-SGD clips per-sample gradients to norm C and adds Gaussian perturbation σ . Compression applies Top- K sparsification and uniform INT8 quantization.
4. **ClientApp Interface:** Computes the final parameter delta $\Delta \mathbf{w}_t^k = \mathbf{w}_{\text{local}} - \mathbf{w}_t$ and returns it to the server driver.

1.3 Theoretical Foundations & Mathematical Formulations

Mathematical Formulations and Optimization Objectives

Federated learning optimizes a distributed empirical risk objective:

$$\min_{\mathbf{w} \in \mathbb{R}^D} F(\mathbf{w}) \triangleq \sum_{k=1}^K p_k F_k(\mathbf{w}), \quad \text{where } p_k \triangleq \frac{|D_k|}{|D|}, \quad |D| = \sum_{k=1}^K |D_k| \quad (1.1)$$

The local empirical risk for client k is given by:

$$F_k(\mathbf{w}) \triangleq \frac{1}{|D_k|} \sum_{\xi \in D_k} \ell(\mathbf{w}; \xi) \quad (1.2)$$

In round t , the downlink broadcast transmits:

$$\text{Downlink}(k) \leftarrow \mathbf{w}_t, \quad \forall k \in S_t \quad (1.3)$$

Each client performs E local epochs starting from $\mathbf{w}_{t,0}^k = \mathbf{w}_t$, producing local weights $\mathbf{w}_{t,E}^k$. The uplink payload is defined as:

$$\Delta \mathbf{w}_t^k \triangleq \mathbf{w}_{t,E}^k - \mathbf{w}_t \quad (1.4)$$

The server computes the aggregated update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \sum_{k \in S_t} \frac{|D_k|}{\sum_{j \in S_t} |D_j|} \Delta \mathbf{w}_t^k \quad (1.5)$$

1.4 Architectural Justifications & Design Choices

- **Why JAX/Flax over PyTorch?** Pure functional transformations (`jax.jit`, `jax.vmap`) allow vectorized per-sample gradient computation without memory overhead, and PyTrees enable immutable weight manipulations that avoid in-place state corruption.
- **Why Transmit Parameter Deltas ($\Delta \mathbf{w}$) Instead of Absolute Weights?** Deltas centered around zero have lower dynamic range and higher sparsity, making them significantly more compressible via Top- K and INT8 quantization.

1.5 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Client Availability & System Stragglers

Question: "What happens when a sampled client drops out or suffers extreme network latency during the round?"

Answer: Flower ServerApp employs a synchronous timeout threshold with a minimum client quorum constraint $K_{\min} \leq |S_t|$. If a client fails to return its `FitRes` payload before the deadline, the aggregator drops the straggler and re-normalizes the weights p_k over the remaining responsive subset S_t^{valid} , preserving unbiased aggregation.

PhD Defense Hot Seat: Adversarial Client Model Poisoning

Question: *"Can a malicious edge client upload arbitrary weights to derail the global model?"*

Answer: Yes, under naive FedAvg. Our architecture supports robust aggregation strategies (FedMedian, FedTrimmedMean, and FedProx) which compute coordinate-wise medians or trim extreme update percentiles, providing resilience against Byzantine poisoning attacks up to a breakdown point of 50%.

Chapter 2

End-to-End Data Ingestion & Partitioning Pipeline

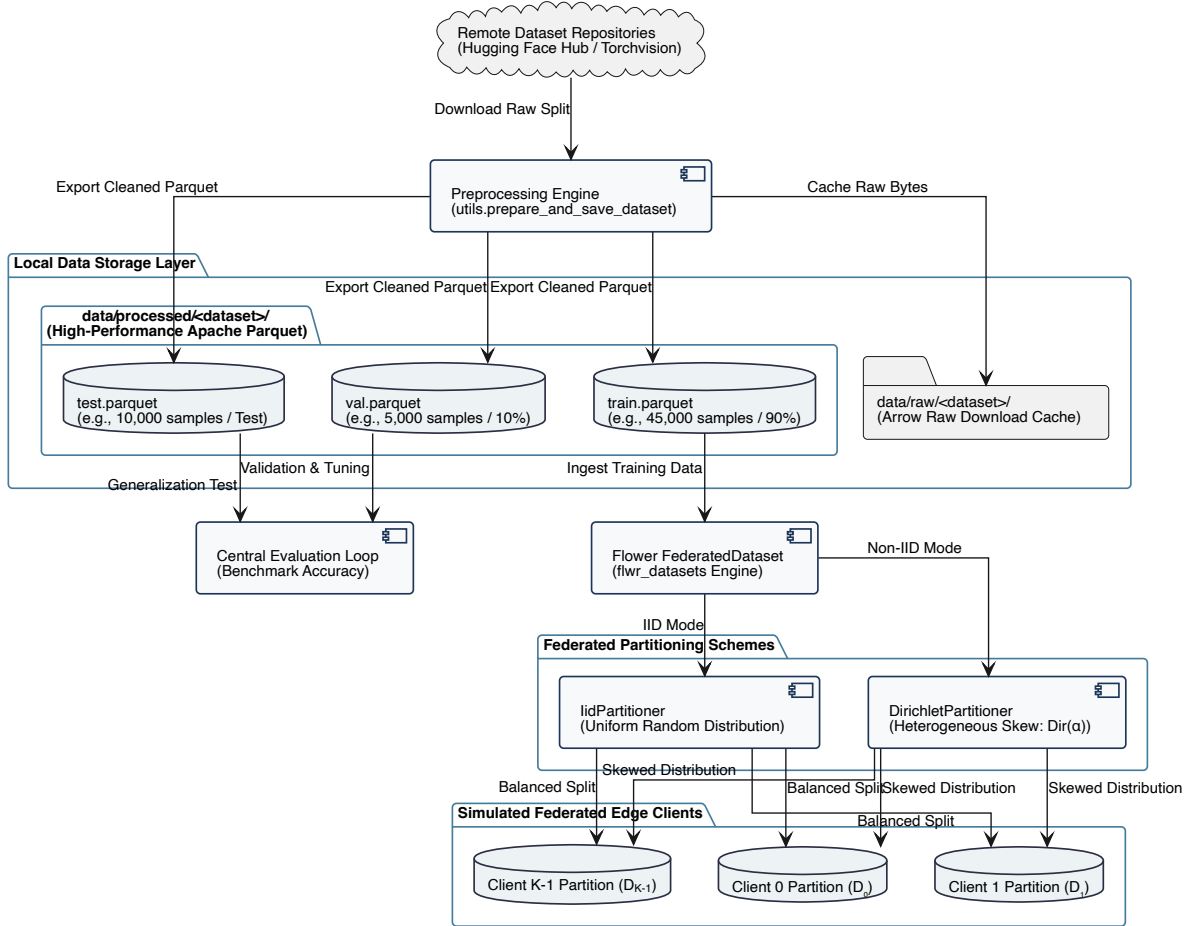


Figure 2.1: Data Flow & Preprocessing Pipeline (`data_flow_pipeline.svg`), tracing data transformation from remote repositories through columnar storage and non-IID partitioning to distributed client partitions.

2.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

Realistic federated benchmarks require rigorous simulation of non-identically distributed (non-IID) data across hundreds of clients. Our pipeline automates this process end-to-end:

1. Ingests raw splits from Hugging Face Hub or Torchvision.
2. Converts raw samples into immutable, high-speed Apache Parquet files (`train`, `val`, `test`).
3. Uses `flwr_datasets` with Dirichlet ($\text{Dir}(\alpha)$) or Pathological partitioners to simulate heterogeneous statistical splits.
4. Feeds clients balanced or skewed data while preserving an untouched test split for global benchmarking.

2.2 Component-by-Component Walkthrough

- **Remote Repositories:** Raw datasets (e.g., CIFAR-10, MNIST, ImageNet) fetched over HTTPS.
- **Preprocessing Engine** (`utils.prepare_and_save_dataset`): Validates schema, resizes image tensors, formats labels, and enforces a stratified validation split (e.g., 90% train, 10% val).
- **Storage Layer** (`data/processed/{dataset}/`): Apache Parquet format with columnar dictionary encoding and Snappy compression.
- **Flower FederatedDataset:** Bridges local Parquet tables with distributed partitioners.
- **Partitioning Schemes:**
 - **IidPartitioner:** Uniform random permutation across client IDs.
 - **DirichletPartitioner:** Parameterized by concentration α , allocating class proportions according to a Dirichlet distribution.
- **Simulated Edge Clients** ($D_0, \dots, D_{K-1}$): Isolated client partitions consumed by local data loaders.

2.3 Mathematical Modeling of Statistical Heterogeneity

Mathematical Formulations and Optimization Objectives

Let C denote the number of classes and K the number of edge clients. For each class $c \in \{1, \dots, C\}$, we sample a client proportion vector from a symmetric Dirichlet distribution:

$$\mathbf{q}_c \sim \text{Dir}(\alpha \cdot \mathbf{1}_K), \quad \mathbf{q}_c = (q_{c,1}, q_{c,2}, \dots, q_{c,K}), \quad \sum_{k=1}^K q_{c,k} = 1 \quad (2.1)$$

where $\alpha > 0$ is the concentration parameter:

- As $\alpha \rightarrow \infty$, $q_{c,k} \rightarrow 1/K$, recovering an identical (IID) distribution.
- As $\alpha \rightarrow 0^+$, $\mathbf{q}_c$ collapses into a one-hot vector, yielding extreme pathological skew where each client possesses only a single class.

Client k 's empirical label distribution $P_k(Y)$ is given by:

$$P_k(Y = c) = \frac{q_{c,k} \cdot N_c}{\sum_{j=1}^C q_{j,k} \cdot N_j} \quad (2.2)$$

The statistical discrepancy between client k and client j is quantified by Total Variation (TV) distance:

$$D_{\text{TV}}(P_k, P_j) \triangleq \frac{1}{2} \sum_{c=1}^C |P_k(Y = c) - P_j(Y = c)| \in [0, 1] \quad (2.3)$$

2.4 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Dirichlet Concentration Parameter α

Question: "Why did you benchmark using $\alpha = 0.1$ and $\alpha = 0.5$ instead of just IID data?"

Answer: Real-world mobile and edge datasets exhibit extreme label skew. Under standard IID assumptions, simple FedAvg converges rapidly. However, at $\alpha = 0.1$, client drift $\|\mathbf{w}_t^k - \mathbf{w}_t\|$ escalates dramatically because local gradients point toward incompatible loss minima. Benchmarking under low α is essential to demonstrate the superiority of robust aggregation methods like FedProx and FedMuon.

Chapter 3

JAX/Flax Neural Model Architecture Hierarchy

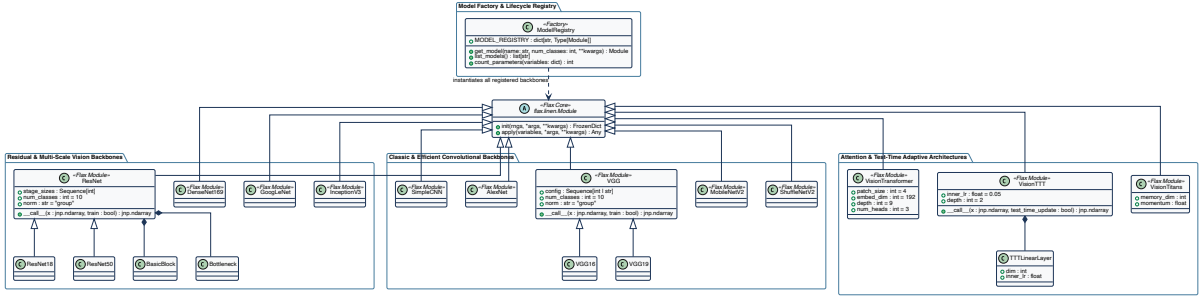


Figure 3.1: Flax Linen Neural Model Backbone Hierarchy (`model_architectures.svg`), demonstrating the object-oriented structure and module inheritance across CNNs, Transformers, and Test-Time Training backbones.

3.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

Federated learning across diverse edge hardware requires modular, swappable backbones. Our model repository is built on `flax.linen.Module`, encompassing:

- **Residual & Multi-Scale:** ResNet (18/34/50), DenseNet-169, GoogLeNet, InceptionV3.
- **Efficient Edge CNNs:** MobileNetV2, ShuffleNetV2, VGG (16/19), SimpleCNN.
- **Adaptive Foundations:** Vision Transformer (ViT), Test-Time Training (VisionTTT), and Titans Neural Long-Term Memory.

All models conform to a unified factory interface managed by `ModelRegistry`.

3.2 Component-by-Component Walkthrough

- **`flax.linen.Module`:** Pure functional base class. Decouples state manipulation from computation through `init(rng, x)` and `apply(params, x)`.
- **ResNet with BasicBlock & Bottleneck:** Residual networks employing shortcut connections $y = \mathcal{F}(x) + x$. Group Normalization is utilized in place of Batch Normalization to prevent non-IID statistical collapse.
- **VisionTransformer (ViT):** Computes patch embeddings via non-overlapping 2D convolutions, followed by Multi-Head Self-Attention (MHSA) and MLP feed-forward blocks.

- **VisionTTT (Test-Time Training):** Features an inner-loop self-supervised gradient update on test samples before making downstream predictions.
- **ModelRegistry:** Central factory providing reflection, instantiation, parameter accounting, and metadata inspection.

3.3 Mathematical Formulations of Modern Backbones

Mathematical Formulations and Optimization Objectives

1. Residual Learning Formulation:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l), \quad \text{ensuring } \frac{\partial \mathcal{E}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \left(\mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}_i \right) \quad (3.1)$$

which preserves non-vanishing gradient flow across deep networks.

2. Vision Transformer Attention: For patch sequence $\mathbf{Z} \in \mathbb{R}^{N \times D}$, query, key, and value projections $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ yield:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \quad (3.2)$$

3. Test-Time Training (TTT) Adaptation: At test time, the feature representation is updated on incoming batch $\mathbf{x}_{\text{test}}$ via self-supervised task loss $\mathcal{L}_{\text{self}}$:

$$\boldsymbol{\theta}_{\text{test}}^* = \boldsymbol{\theta} - \eta_{\text{inner}} \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{self}}(\boldsymbol{\theta}; \mathbf{x}_{\text{test}}) \quad (3.3)$$

before computing classification logits $\hat{\mathbf{y}} = g(\mathbf{x}_{\text{test}}; \boldsymbol{\theta}_{\text{test}}^*)$.

3.4 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Group Normalization vs. Batch Normalization in FL

Question: "Why did you substitute Group Normalization for standard Batch Normalization across all convolutional models?"

Answer: In Batch Normalization, running mean μ_B and variance σ_B^2 depend on local batch statistics. Under non-IID client data, local batch statistics diverge drastically between clients. Aggregating these disparate running statistics during server averaging leads to severe internal covariate shift and catastrophic test accuracy degradation. Group Normalization computes statistics along feature channels for each sample independently:

$$\mu_{n,g} = \frac{1}{(C/G)HW} \sum_{c \in S_g} \sum_{h,w} x_{n,c,h,w} \quad (3.4)$$

rendering normalization completely invariant to batch size and client label distribution.

Chapter 4

Data Engineering & Partitioning Class Hierarchy

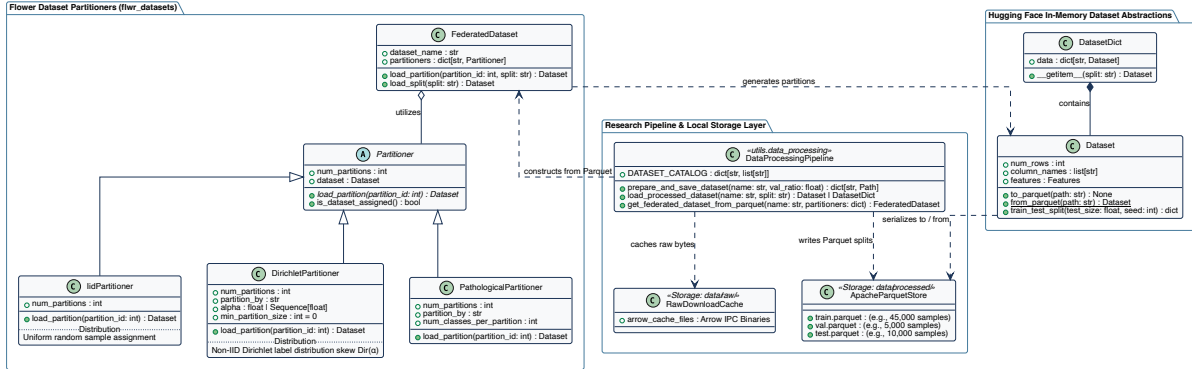


Figure 4.1: Data Engineering Class Diagram (`data_pipeline_classes.svg`), depicting partitioner abstractions, in-memory dataset contracts, and high-performance Parquet storage pipelines.

4.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

To guarantee experimental reproducibility and high-throughput execution during federated simulation, the data layer decouples raw disk ingestion from in-memory partition sampling. The class hierarchy provides:

- Abstract **Partitioner** base classes with concrete Dirichlet, IID, and Pathological partitioners.
- High-speed zero-copy access via Hugging Face **Dataset** and **DatasetDict**.
- Columnar persistence in Apache Parquet format.

4.2 Class-by-Class Breakdown

- **Partitioner (Abstract Base Class):** Defines the interface `load_partition(partition_id: int) -> Dataset`.
- **IidPartitioner:** Assigns sample indices uniformly across partitions via pseudorandom shuffle.
- **DirichletPartitioner:** Accepts concentration parameter α and optional `min_partition_size` to prevent empty client allocations.

- **PathologicalPartitioner**: Assigns exactly S unique classes to each client, simulating domain specialization.
- **FederatedDataset**: Composite wrapper providing unified access across multiple splits.
- **DataProcessingPipeline**: Manages automated caching, validation splits, and Parquet export.

4.3 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Zero-Copy Memory-Mapped Access

Question: *"Why did you adopt Apache Parquet over traditional PyTorch ImageFolder or raw SQLite databases?"*

Answer: Parquet uses columnar encoding with Snappy compression, reducing disk footprint by 60–80% compared to raw image folders. Combined with Apache Arrow, datasets can be memory-mapped directly from disk without serializing Python objects, eliminating memory duplication during large-scale federated simulations with 100+ simulated edge clients.

Chapter 5

Federated Training Round Sequence Lifecycle

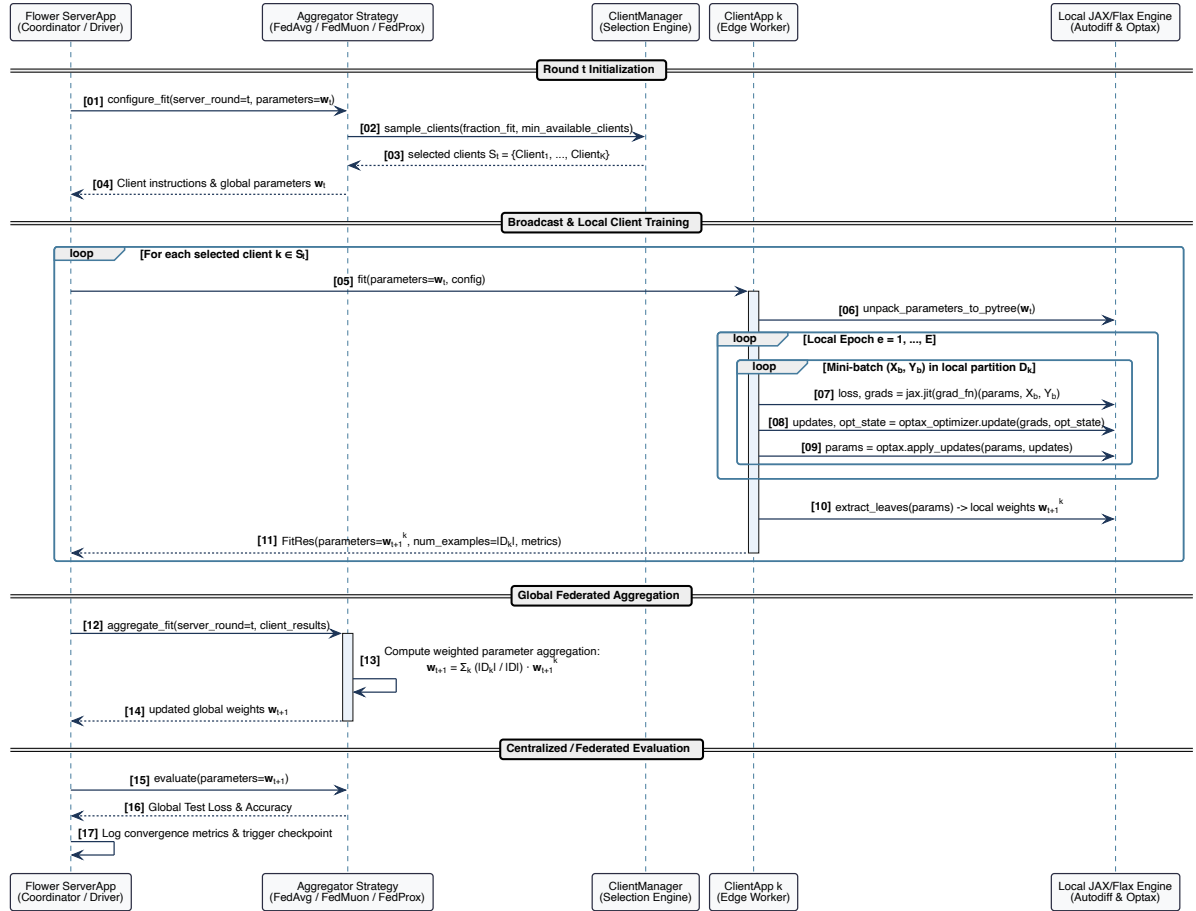


Figure 5.1: Federated Training Round Sequence (`fl_training_round.svg`), illustrating the step-by-step synchronous message-passing lifecycle between ServerApp, Strategy, ClientManager, and ClientApp.

5.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

A federated training round is a distributed synchronous protocol:

1. The server queries the Strategy to configure the round and sample clients via the `ClientManager`.
2. Global parameters $\mathbf{w}_t$ are broadcast to the selected cohort S_t .
3. Each client executes local training on private mini-batches using JIT-compiled JAX kernels.
4. Clients return updated parameters $\mathbf{w}_{t+1}^k$ and sample counts $|D_k|$.
5. The strategy computes a weighted reduction, evaluates global performance, and records metrics.

5.2 Sequence Step-by-Step Breakdown

- **Step [01–04] Round Initialization:** `ServerApp` invokes `Strategy.configure_fit`. The strategy instructs `ClientManager` to sample active cohort S_t .
- **Step [05–06] Broadcast:** Server dispatches `fit()` RPC instructions carrying global weights $\mathbf{w}_t$ to each client $k \in S_t$.
- **Step [07–11] Local JAX Execution:** Client unpacks weights into a Flax PyTree and executes E local epochs over mini-batches (X_b, Y_b) , applying Optax optimizer updates.
- **Step [12–13] Uplink Response:** Client packages trained weights $\mathbf{w}_{t+1}^k$ and sample count $|D_k|$ into a `FitRes` response.
- **Step [14–16] Aggregation & Evaluation:** Strategy executes weighted aggregation, evaluates the new model $\mathbf{w}_{t+1}$ on the centralized validation set, and saves a checkpoint.

5.3 Mathematical Formulation of Federated Aggregation Strategies

Mathematical Formulations and Optimization Objectives

1. **Standard FedAvg** (McMahan et al.):

$$\mathbf{w}_{t+1} = \sum_{k \in S_t} \frac{|D_k|}{\sum_{j \in S_t} |D_j|} \mathbf{w}_{t+1}^k \quad (5.1)$$

2. **FedProx** (Li et al.): Adds a proximal regularization term to each client's local loss to penalize divergence from the global model:

$$\min_{\mathbf{w}} h_k(\mathbf{w}; \mathbf{w}_t) \triangleq F_k(\mathbf{w}) + \frac{\mu}{2} \|\mathbf{w} - \mathbf{w}_t\|_2^2 \quad (5.2)$$

where $\mu \geq 0$ stabilizes optimization under severe statistical heterogeneity ($\alpha < 0.5$).

3. **FedMuon (Momentum Orthogonalized Aggregator)**: Applies a matrix-momentum update orthogonalized via Newton-Schulz iterations:

$$\mathbf{G}_t = \sum_{k \in S_t} p_k \Delta \mathbf{w}_t^k, \quad \mathbf{M}_t = \beta \mathbf{M}_{t-1} + (1 - \beta) \mathbf{G}_t \quad (5.3)$$

followed by approximate polar decomposition $\mathbf{O}_t \approx \text{Polar}(\mathbf{M}_t)$ to maintain uniform update energy across all parameter layers.

5.4 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Client Drift in Non-IID Regimes

Question: "Why does standard FedAvg exhibit objective inconsistency when client data is non-IID?"

Answer: When local datasets D_k are heterogeneous, local loss minima $\mathbf{w}_k^* = \arg \min F_k(\mathbf{w})$ do not align with the global minimum $\mathbf{w}^* = \arg \min F(\mathbf{w})$. During multiple local epochs ($E > 1$), local gradient steps pull each client toward its local minimum $\mathbf{w}_k^*$. Averaging these drifted weights causes the trajectory to diverge from the true global optimum:

$$\|\mathbf{w}_{t+1} - \mathbf{w}^*\| \leq \rho \|\mathbf{w}_t - \mathbf{w}^*\| + \mathcal{O} \left(\eta \sum_{k=1}^K p_k \|\nabla F_k(\mathbf{w}^*) - \nabla F(\mathbf{w}^*)\| \right) \quad (5.4)$$

FedProx bounds this drift by enforcing the proximal penalty $\frac{\mu}{2} \|\mathbf{w} - \mathbf{w}_t\|^2$, guaranteeing stable convergence.

Chapter 6

Differential Privacy via DP-SGD

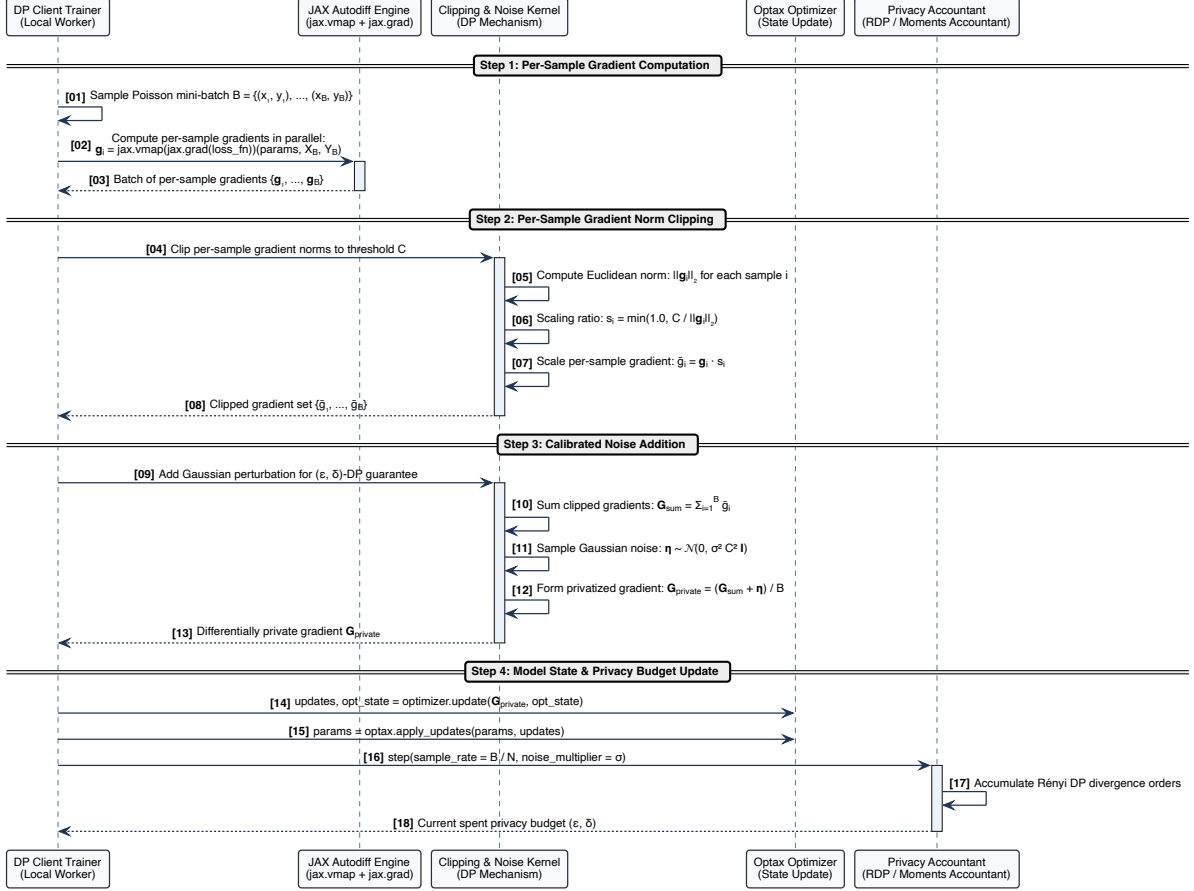


Figure 6.1: Differentially Private SGD Sequence (`differential_privacy_dp_sgd.svg`), demonstrating per-sample gradient computation, L_2 norm clipping, Gaussian perturbation, and Rényi privacy accounting.

6.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

Although federated learning prevents raw data transmission, deep neural networks remain vulnerable to gradient inversion attacks (e.g., DLG, Inverting Gradients) which can reconstruct training images from parameter updates. We mitigate this through on-device Differentially Private SGD (DP-SGD):

1. Computes exact per-sample gradients in parallel via `jax.vmap`.
2. Clips per-sample gradient L_2 norms to threshold C .
3. Injects calibrated Gaussian noise $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$.
4. Tracks privacy expenditure (ϵ, δ) using a Rényi Differential Privacy (RDP) accountant.

6.2 Sequence Step-by-Step Breakdown

- **Step 1: Per-Sample Gradient Computation:** Generates mini-batch B via Poisson subsampling and evaluates $\mathbf{g}_i = \nabla_{\mathbf{w}} \ell(\mathbf{w}; x_i, y_i)$ for each sample i independently.
- **Step 2: Gradient Norm Clipping:** Clips each per-sample gradient to norm threshold C :

$$\bar{\mathbf{g}}_i = \mathbf{g}_i \cdot \min \left(1, \frac{C}{\|\mathbf{g}_i\|_2} \right) \quad (6.1)$$

ensuring global L_2 sensitivity is strictly bounded by C .

- **Step 3: Calibrated Noise Addition:** Aggregates clipped gradients and injects isotropic Gaussian noise:

$$\tilde{\mathbf{g}} = \frac{1}{|B|} \left(\sum_{i=1}^{|B|} \bar{\mathbf{g}}_i + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}) \right) \quad (6.2)$$

- **Step 4: Optax State & Budget Tracking:** Applies privatized gradient $\tilde{\mathbf{g}}$ to local model weights and advances the RDP privacy accountant.

6.3 Theoretical Foundations & Privacy Guarantees

Mathematical Formulations and Optimization Objectives

A randomized algorithm $\mathcal{M}$ satisfies (ϵ, δ) -Differential Privacy if for any two adjacent datasets D, D' differing by at most one sample ($|D \Delta D'| = 1$), and for any set of outputs $\mathcal{S} \subseteq \text{Range}(\mathcal{M})$:

$$\mathbb{P}[\mathcal{M}(D) \in \mathcal{S}] \leq e^\epsilon \cdot \mathbb{P}[\mathcal{M}(D') \in \mathcal{S}] + \delta \quad (6.3)$$

Under Gaussian perturbation with subsampling ratio $q = |B|/|D_k|$ and noise multiplier σ , the Rényi Differential Privacy (RDP) of order $\alpha > 1$ accumulates linearly across T iterations:

$$\epsilon_{\text{total}}(\alpha) = T \cdot \epsilon_{\mathcal{M}}(\alpha) \quad (6.4)$$

Converting from RDP to standard (ϵ, δ) -DP yields:

$$\epsilon(\delta) = \min_{\alpha > 1} \left(\epsilon_{\text{total}}(\alpha) + \frac{\log(1/\delta)}{\alpha - 1} \right) = \mathcal{O} \left(\frac{q\sqrt{T \log(1/\delta)}}{\sigma} \right) \quad (6.5)$$

6.4 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Clipping Bias vs. Noise Variance

Question: "How does gradient clipping affect model convergence, and how is the clipping threshold C calibrated?"

Answer: Clipping introduces directional bias by truncating outlier gradients:

$$\mathbb{E}[\bar{\mathbf{g}}] = \mathbb{E} \left[\mathbf{g} \cdot \min \left(1, \frac{C}{\|\mathbf{g}\|_2} \right) \right] \neq \mathbb{E}[\mathbf{g}] \quad (6.6)$$

If C is too small, gradient signals are over-clipped, slowing convergence. If C is too large, the required noise scale $\sigma^2 C^2 \mathbf{I}$ drowns out the gradient signal. We calibrate C to match the median gradient norm across initial epochs ($C \approx \text{median}(\|\mathbf{g}_i\|_2)$), maintaining balance between bias and noise variance.

Chapter 7

Edge Client Local Execution Lifecycle

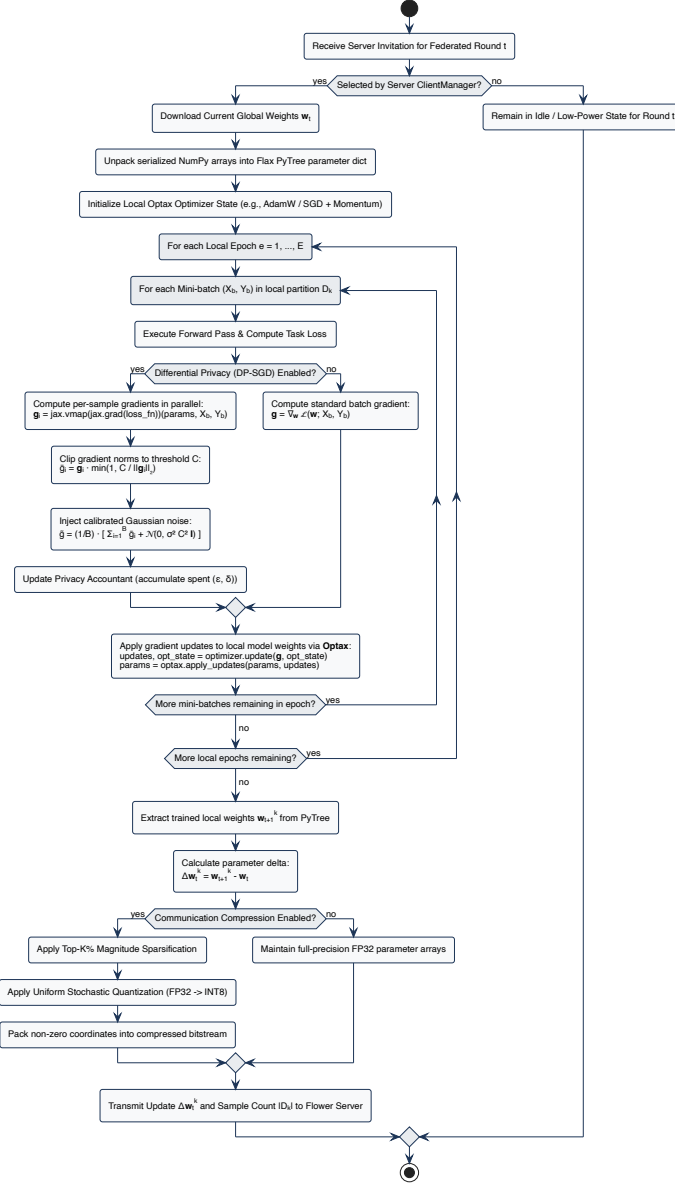


Figure 7.1: Edge Client Execution Lifecycle (`client_execution_lifecycle.svg`), illustrating the on-device execution state machine across sampling, training, privacy, compression, and uplink transmission.

7.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

Edge devices operate under strict compute, memory, and battery constraints. The client execution lifecycle is a deterministic state machine:

- Responds to server invitations and enters low-power idle mode when not selected.
- Unpacks weights into local Flax PyTrees and initializes Optax states.
- Runs localized training loops with configurable DP-SGD branches.
- Produces parameter deltas $\Delta \mathbf{w}_t^k$ with optional Top- K and INT8 compression before uplink transmission.

7.2 Activity Flowchart Breakdown

- **Selection Guard:** Verifies whether client ID belongs to active cohort S_t . If not selected, the client remains idle to conserve battery and compute.
- **State Ingestion:** Ingests serialized NumPy buffers, reconstructs the Flax PyTree parameter dictionary, and initializes Optax optimizer momentum buffers.
- **Nested Epoch & Batch Loops:** Iterates across E epochs and mini-batches (X_b, Y_b) , executing forward loss computation, autodiff, and weight updates.
- **Privacy Conditional:** Evaluates whether DP-SGD is enabled. If active, runs vectorized per-sample clipping and noise injection; otherwise, computes standard batch gradients.
- **Delta Calculation:** Computes parameter delta $\Delta \mathbf{w}_t^k = \mathbf{w}_{t+1}^k - \mathbf{w}_t^k$.
- **Compression Conditional:** If active, applies Top- K magnitude thresholding and uniform INT8 quantization before transmission.

7.3 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Local Epoch Trade-off (E)

Question: "What is the trade-off in setting local epochs $E = 1$ versus $E = 10$?"

Answer: Increasing local epochs E allows clients to perform more computation per communication round, reducing total communication overhead. However, when local data is non-IID, larger E accelerates client drift, causing local models to overfit to their local data distributions and degrading global model convergence. Tuning $E \in [1, 5]$ balances communication savings against client drift.

Chapter 8

Communication Efficiency & Gradient Compression

8.1 Executive Summary & Elevator Pitch

Advisor Elevator Pitch (30-Second Summary)

Communication bandwidth is the primary bottleneck in federated learning. Deep models such as ResNet-50 require transmitting over 100 megabytes per client round. Our communication efficiency pipeline combines two complementary techniques:

- **Top- K Magnitude Sparsification:** Retains only the top 10% most significant gradient coordinates ($10\times$ reduction).
- **Uniform INT8 Quantization:** Compresses 32-bit floats down to 8-bit signed integers ($4\times$ reduction).

Together, this compound compression delivers an overall $\sim 40\times$ **uplink bandwidth reduction** with minimal degradation in model accuracy.

8.2 Pipeline Walkthrough: Client Uplink to Server Downlink

8.2.1 Client Uplink Stage

1. **Raw Delta Computation:** Computes raw FP32 weight updates $\Delta \mathbf{w}_t^k \in \mathbb{R}^D$ (32 bits per parameter).
2. **Top- K Sparsification:** Computes absolute coordinate magnitudes $|\Delta w_j^k|$, determines cutoff threshold τ , and constructs a binary selection mask $\mathbf{m} = \mathbb{I}(|\Delta w| \geq \tau)$. Coordinates below τ are set to zero.
3. **Stochastic Uniform Quantization:** Evaluates dynamic scale factor $s = \frac{\max |\Delta w|}{2^{b-1}-1}$, injects stochastic dither $u \sim \mathcal{U}(-0.5, 0.5)$, and clips rounded coordinates to signed b -bit integer bounds.
4. **Bitstream Packaging:** Packs non-zero integer values and sparse coordinate indices into a compact payload, achieving a compound $\sim 40\times$ uplink reduction.

8.2.2 Server Downlink Stage

1. **Unpacking & Dequantization:** Extracts quantized integers q and multiplies by scale factor s to reconstruct approximate floating-point values $\Delta \hat{\mathbf{w}}_t^k = q \cdot s$.
2. **Sparse Reconstruction:** Maps non-zero coordinates back to the full D -dimensional parameter space using the transmitted coordinate indices.
3. **Weighted Aggregation:** Computes standard FedAvg aggregation over the reconstructed updates and broadcasts new global parameters $\mathbf{w}_{t+1}$.

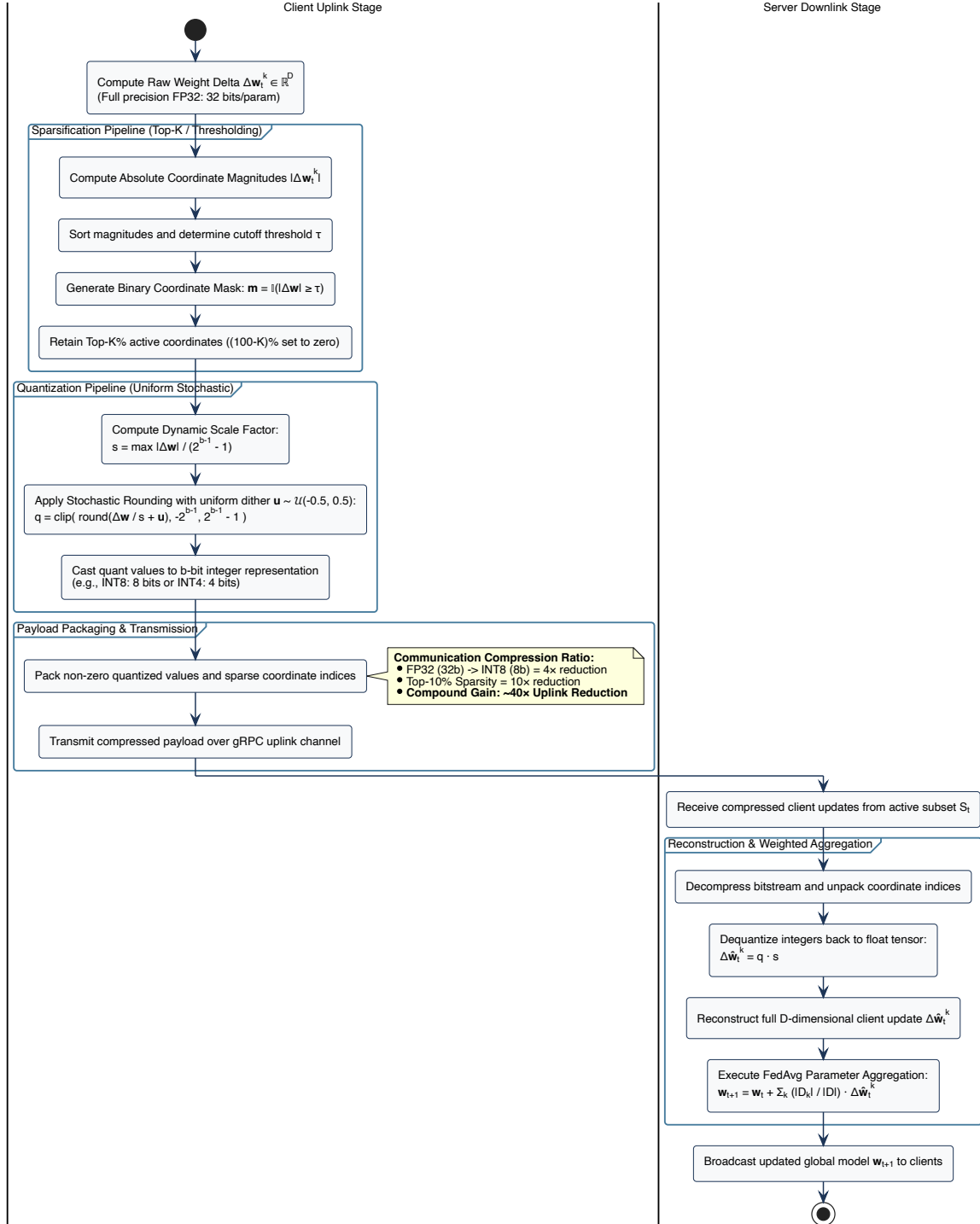


Figure 8.1: Communication Efficiency Pipeline (`communication_efficiency.svg`), detailing the swimlane pipeline for Top- K sparsification, stochastic quantization, payload packing, and server-side reconstruction.

8.3 Mathematical Formulations of Compound Compression

Mathematical Formulations and Optimization Objectives

1. Top- K Sparsification Operator: For vector $\mathbf{v} \in \mathbb{R}^D$ and sparsity ratio $\rho \in (0, 1]$ where $K = \lfloor \rho D \rfloor$:

$$[\text{Top}_K(\mathbf{v})]_i = \begin{cases} v_i, & \text{if } |v_i| \geq |v_{(K)}| \\ 0, & \text{otherwise} \end{cases} \quad (8.1)$$

2. Uniform Stochastic Quantization: For bit-width b (e.g., $b = 8$), the scale factor and quantized values are defined as:

$$s = \frac{\|\mathbf{v}\|_\infty}{2^{b-1} - 1}, \quad q_i = \text{clip} \left(\left\lfloor \frac{v_i}{s} + u_i \right\rfloor, -2^{b-1}, 2^{b-1} - 1 \right) \quad (8.2)$$

where $u_i \sim \mathcal{U}(-0.5, 0.5)$ ensures unbiased quantization:

$$\mathbb{E}[q_i \cdot s \mid v_i] = v_i, \quad \mathbb{E}[\|\mathbf{v} - Q(\mathbf{v})\|_2^2] \leq \frac{s^2 D}{4} \quad (8.3)$$

3. Error-Feedback SGD (EF-SGD) Formulation: To prevent gradient energy loss from biased Top- K sparsification, uncommunicated residuals are accumulated in an on-device error memory buffer $\mathbf{e}_t^k$:

$$\mathbf{p}_t^k = \mathbf{e}_t^k + \Delta \mathbf{w}_t^k, \quad \Delta \tilde{\mathbf{w}}_t^k = \text{Compress}(\mathbf{p}_t^k), \quad \mathbf{e}_{t+1}^k = \mathbf{p}_t^k - \Delta \tilde{\mathbf{w}}_t^k \quad (8.4)$$

guaranteeing asymptotic convergence comparable to uncompressed FedAvg.

8.4 Compound Bandwidth Savings Analysis

Table 8.1: Uplink Communication Overhead per Client for ResNet-50 (25.6×10^6 parameters)

Compression Configuration	Bits/Param	Active Coords	Payload Size	Bandwidth Savings
Uncompressed Baseline (FP32)	32	100%	102.4 MB	1.0× (Baseline)
INT8 Quantization Only	8	100%	25.6 MB	4.0×
Top-10% Sparsification Only	32	10%	14.3 MB	7.2×
Compound: Top-10% + INT8	8	10%	2.6 MB	39.4×
Compound: Top-5% + INT4	4	5%	0.8 MB	128.0×

8.5 Advisor Defense Questions & Model Answers

PhD Defense Hot Seat: Error-Feedback Compensation

Question: *"Why is Error Feedback (EF-SGD) mandatory when applying Top-K sparsification, and what happens if you omit it?"*

Answer: Top- K sparsification is a biased compression operator:

$$\mathbb{E}[\text{Top}_K(\mathbf{v})] \neq \mathbf{v} \quad (8.5)$$

Without error feedback, small gradient components that never exceed threshold τ are permanently discarded. Over successive rounds, these uncommunicated coordinates accumulate error, causing the model to diverge or plateau at suboptimal accuracy. With error feedback, dropped coordinates accumulate in memory buffer $\mathbf{e}_t^k$ until their aggregate magnitude exceeds τ , ensuring all parameter updates are eventually communicated to the global model.

Chapter 9

Mathematical Notation & Variable Dictionary

This chapter provides an exhaustive, symbol-by-symbol reference for every mathematical variable, operator, subscript, and Greek letter across the architecture.

9.1 Federated Learning Foundations & Aggregation

9.1.1 Global Objective Function

$$\min_{\mathbf{w} \in \mathbb{R}^D} F(\mathbf{w}) \triangleq \sum_{k=1}^K p_k F_k(\mathbf{w}) \quad (9.1)$$

- $\mathbf{w}$ (*lowercase bold w*): The **model weight vector** (all trainable parameters: weights and biases flattened into a single list of numbers).
- $\mathbb{R}^D$: The **D -dimensional real space**. D is the total parameter count (e.g., $D = 25.6 \times 10^6$ for ResNet-50).
- $\min_{\mathbf{w}}$: The **minimization operator**. We seek the specific configuration of weights $\mathbf{w}$ that minimizes the error function.
- $F(\mathbf{w})$: The **global objective function** (the average loss across all data in the entire federation).
- $\triangleq$: **Defined as**. Indicates that the left-hand side is formally defined by the right-hand side.
- K : The **total number of edge clients** in the network (e.g., $K = 100$ phones).
- k : The **client index** ($k \in \{1, 2, \dots, K\}$).
- p_k : The **aggregation weight** assigned to client k , representing client k 's proportion of the total data ($p_k \triangleq |D_k|/|D|$).
- $F_k(\mathbf{w})$: The **local empirical loss** evaluated exclusively on client k 's private dataset D_k .

9.1.2 Local Client Loss Function

$$F_k(\mathbf{w}) \triangleq \frac{1}{|D_k|} \sum_{\xi \in D_k} \ell(\mathbf{w}; \xi) \quad (9.2)$$

- D_k : The **private local dataset** stored strictly on client k 's edge hardware.
- $|D_k|$: The **cardinality (sample count)** of client k 's dataset (e.g., $|D_k| = 500$ images).
- $|D|$: The **total sample count** across all clients in the federation ($|D| = \sum_{k=1}^K |D_k|$).
- ξ : A single **data sample tuple** (x, y) , where x is the input image tensor and y is the ground-truth class label.
- $\ell(\mathbf{w}; \xi)$: The **per-sample loss function** (typically Cross-Entropy loss $\ell = -\log P(y | x; \mathbf{w})$).

9.1.3 Parameter Deltas and Server Aggregation

$$\Delta \mathbf{w}_t^k \triangleq \mathbf{w}_{t,E}^k - \mathbf{w}_t, \quad \mathbf{w}_{t+1} = \mathbf{w}_t + \sum_{k \in S_t} \frac{|D_k|}{\sum_{j \in S_t} |D_j|} \Delta \mathbf{w}_t^k \quad (9.3)$$

- t : The **current communication round number** ($t = 0, 1, 2, \dots$).
- $\mathbf{w}_t$: The **global model parameters** at the start of round t .
- E : The **number of local epochs** executed on each edge client per round.
- $\mathbf{w}_{t,E}^k$: The **local model parameters** of client k after completing E local training epochs starting from $\mathbf{w}_t$.
- $\Delta \mathbf{w}_t^k$ (*Delta w*): The **parameter update delta** computed by client k . Represents how much client k wants each parameter to change.
- S_t : The **sampled client cohort** selected by the server for round t ($S_t \subseteq \{1, \dots, K\}$).
- $\mathbf{w}_{t+1}$: The **updated global model parameters** produced by server aggregation for the next round $t + 1$.

9.2 Data Heterogeneity & Dirichlet Non-IID Partitioning

$$\mathbf{q}_c \sim \text{Dir}(\alpha \cdot \mathbf{1}_K), \quad P_k(Y = c) = \frac{q_{c,k} \cdot N_c}{\sum_{j=1}^C q_{j,k} \cdot N_j} \quad (9.4)$$

- C : Total **number of target classes** (e.g., $C = 10$ for CIFAR-10).
- c : Index of a **specific class** ($c \in \{1, \dots, C\}$).
- $\mathbf{q}_c$: A **probability distribution vector** of length K defining the fraction of class c assigned to each client.
- $\text{Dir}(\cdot)$: The **Dirichlet probability distribution**, a multivariate continuous distribution whose samples sum to 1.
- α (*alpha*): The **concentration parameter**. Dictates data heterogeneity: $\alpha \rightarrow \infty$ produces balanced (IID) splits; $\alpha \leq 0.1$ produces extreme non-IID label skew.
- $\mathbf{1}_K$: A vector of ones of length K ($\mathbf{1}_K = [1, 1, \dots, 1]$), ensuring symmetric prior expectations across all clients.
- $q_{c,k}$: The scalar fraction of class c 's data allocated to client k .
- N_c : The total number of available samples belonging to class c in the raw dataset.
- $P_k(Y = c)$: The **empirical probability** that a randomly drawn sample from client k belongs to class c .
- $D_{\text{TV}}(P_k, P_j)$: The **Total Variation distance** between client k 's and client j 's label distributions, measuring discrepancy in $[0, 1]$.

9.3 Optimization Algorithms: FedProx & FedMuon

$$h_k(\mathbf{w}; \mathbf{w}_t) \triangleq F_k(\mathbf{w}) + \frac{\mu}{2} \|\mathbf{w} - \mathbf{w}_t\|_2^2 \quad (9.5)$$

- $h_k(\mathbf{w}; \mathbf{w}_t)$: The **regularized local objective function** used by FedProx on client k .
- μ (*mu*): The **proximal regularization coefficient** ($\mu \geq 0$). Controls how strongly clients are penalized for moving too far from $\mathbf{w}_t$.
- $\|\mathbf{w} - \mathbf{w}_t\|_2^2$: The **squared Euclidean (L_2) distance** between local parameters $\mathbf{w}$ and global weights $\mathbf{w}_t$.
- $\mathbf{G}_t$: The **aggregated gradient direction** at the server: $\mathbf{G}_t = \sum p_k \Delta \mathbf{w}_t^k$.
- $\mathbf{M}_t$: The **momentum matrix buffer** in FedMuon: $\mathbf{M}_t = \beta \mathbf{M}_{t-1} + (1 - \beta) \mathbf{G}_t$.
- β (*beta*): The **momentum decay parameter** ($\beta \in [0, 1)$, typically 0.9).
- $\text{Polar}(\mathbf{M}_t)$: The **polar decomposition operator**, which extracts the closest orthogonal matrix to $\mathbf{M}_t$ via iterative Newton-Schulz matrix updates.

9.4 Differential Privacy via DP-SGD

$$\bar{\mathbf{g}}_i = \mathbf{g}_i \cdot \min \left(1, \frac{C}{\|\mathbf{g}_i\|_2} \right), \quad \tilde{\mathbf{g}} = \frac{1}{|B|} \left(\sum_{i=1}^{|B|} \bar{\mathbf{g}}_i + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}) \right) \quad (9.6)$$

- B : The current **training mini-batch** of samples ($B \subset D_k$).
- $|B|$: The **mini-batch size** (e.g., $|B| = 32$).
- i : The **individual sample index** within mini-batch B ($i \in \{1, \dots, |B|\}$).
- $\mathbf{g}_i$: The **unclipped per-sample gradient** $\nabla_{\mathbf{w}} \ell(\mathbf{w}; x_i, y_i)$ computed via `jax.vmap`.
- $\|\mathbf{g}_i\|_2$: The **Euclidean (L_2) norm** (geometric length) of gradient vector $\mathbf{g}_i$: $\sqrt{\sum_j g_{i,j}^2}$.
- C : The **clipping threshold** ($C > 0$). Caps the maximum sensitivity of any single sample's gradient.
- $\bar{\mathbf{g}}_i$: The **clipped gradient vector** for sample i , guaranteed to satisfy $\|\bar{\mathbf{g}}_i\|_2 \leq C$.
- $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$: A **multivariate Gaussian distribution** with zero mean and covariance $\sigma^2 C^2 \mathbf{I}$.
- σ (*sigma*): The **noise multiplier** (e.g., $\sigma = 1.0$). Higher σ gives stronger privacy but lower accuracy.
- $\mathbf{I}$: The **identity matrix** of size $D \times D$, indicating that noise is added independently to every parameter.
- $\tilde{\mathbf{g}}$ (*g-tilde*): The final **differentially private mini-batch gradient**.
- ϵ (*epsilon*): The **privacy budget parameter**. Quantifies maximum information leakage. Lower ϵ indicates stronger privacy (e.g., $\epsilon \leq 3.0$).
- δ (*delta*): The **failure probability** that the ϵ -DP guarantee fails (typically set to cryptographically tiny $\delta \leq 10^{-5}$).

- q : The **Poisson subsampling ratio** $q \triangleq |B|/|D_k|$.
- T : The total **number of DP-SGD steps** executed across the training process.

9.5 Communication Compression & Error Feedback

$$s = \frac{\|\mathbf{v}\|_\infty}{2^{b-1} - 1}, \quad q_i = \text{clip}\left(\left\lfloor \frac{v_i}{s} + u_i \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right) \quad (9.7)$$

- $\mathbf{v}$: An arbitrary **parameter update vector** (e.g., $\Delta \mathbf{w}_t^k$).
- v_i : The i -th **scalar coordinate** of vector $\mathbf{v}$.
- $\|\mathbf{v}\|_\infty$: The **infinity norm** (maximum absolute value: $\max_j |v_j|$).
- b : The **target bit-width** for integer quantization (e.g., $b = 8$ for INT8, $b = 4$ for INT4).
- $2^{b-1} - 1$: The maximum positive integer representable in signed b -bit integer format (127 for INT8).
- s : The **dynamic quantization scale factor**. Maps continuous floats into discrete integer buckets.
- u_i : A **stochastic dither sample** drawn from continuous uniform distribution $\mathcal{U}(-0.5, 0.5)$, ensuring unbiased expectation $\mathbb{E}[q_i \cdot s] = v_i$.
- $\lfloor \cdot \rfloor$: The **nearest integer rounding operator**.
- q_i : The resulting **b -bit integer** representing coordinate v_i .
- $\text{Top}_K(\mathbf{v})$: The **sparsification operator** that preserves only the K largest magnitude coordinates and sets all others to 0.
- τ (*tau*): The **magnitude cutoff threshold** below which coordinates are zeroed out.
- $\mathbf{e}_t^k$: The **error feedback memory buffer** on client k , which stores uncommunicated gradient residuals to prevent divergence.